



Formando nuevas generaciones con sello de excelencia comprometidos con la transformación social de las regiones y un país en paz

Miniestación Meteorológica Programada

Desarrollo de una Miniestación Meteorológica constituida por Sensores y Materiales Reciclables, mediante el Paradigma de Programación Orientada a Objetos (POO)

Emanuel Josue Torres Marquez

Giomar Oneith Moreno Quintero

Loren Sofía Patarroyo Preciado

Walexis Snideer Escalante Villamizar

Universidad de Pamplona

Facultad de Ingenierías y Arquitecturas

Fundamentos de Programación Orientada a Objetos

168411 - Grupo C

Jesus Yndriago

19/06/2026



Miniestación Meteorológica Programada

E. Torres, G. Moreno, L. Patarroyo, W. Escalante

I. Resumen

El presente informe abarca el diseño e implementación de una miniestación meteorológica automatizada programada en un microcontrolador ESP32, capaz de monitorear variables ambientales como temperatura, humedad relativa, humedad del suelo, iluminación y calidad del aire empleando los sensores electrónicos DHT11, HL-69, LDR y MQ-135; junto con la adición de una pantalla LCD, el módulo de comunicación I2C y un par de pulsadores para la visualización de datos en tiempo real.

El desarrollo del software del proyecto se realizó utilizando la metodología de diseño orientado a objetos, aplicando los conceptos y fundamentos propios de este paradigma de programación y desarrollando el software en Visual Studio Code mediante la extensión PlatformIO y el framework Arduino IDE, organizando el proyecto en un diseño modular con archivos de encabezado e implementación para mejorar la organización y el mantenimiento del código.

La parte física del proyecto tuvo en cuenta la creación de una placa de circuito impreso y reutilización de materiales reciclables y sobrantes de otros proyectos, minimizando el gasto económico y la huella ambiental de su producción.

II. Palabras claves

- Estación Meteorológica
- Arduino
- Sensores
- Programación Modular
- Programación Orientada a Objetos (POO)

III. Abstract

This report covers the design and implementation of an automated mini-weather station programmed on an ESP32 microcontroller, capable of monitoring environmental variables such as temperature, relative humidity, soil moisture, light, and air quality using

DHT11, HL-69, LDR, and MQ-135 electronic sensors; along with the addition of an LCD screen, an I2C communication module, and a pair of push buttons for real-time data display.

The project's software was developed using object-oriented design methodology, applying the concepts and fundamentals of this programming paradigm and developing the software in Visual Studio Code using the PlatformIO extension and the Arduino IDE framework, organizing the project into a modular design with header and implementation files to improve code organization and maintainability.

The hardware component of the project involved designing a printed circuit board and reusing recyclable materials and leftover parts from other projects, thereby minimizing both the financial cost and the environmental footprint of its production.

IV. Key Words

- Weather Station
- Arduino
- Sensors
- Modular Programming
- Object-Oriented Programming (OOP)

V. Introducción

En la actualidad, el monitoreo de variables ambientales desempeña un papel fundamental en diferentes áreas como la agricultura, la industria y la investigación, ya que permite conocer las condiciones del entorno y tomar decisiones oportunas basadas en datos reales analizados. Parámetros como la temperatura, la humedad, la iluminación y la calidad del aire influyen directamente en diversos procesos de todo tipo, por lo que resulta importante contar con sistemas capaces de medirlos de forma continua y confiable.

Gracias al avance de los sistemas y de los microcontroladores, es posible desarrollar soluciones de bajo costo para la adquisición y visualización de información ambiental en tiempo real. Entre estas



tecnologías se destaca el ESP32, una plataforma ampliamente utilizada por su capacidad de procesamiento, facilidad de programación y compatibilidad con una gran variedad de sensores electrónicos.

En este contexto, el presente informe abarca el desarrollo de una miniestación meteorológica automatizada capaz de monitorear variables ambientales como la temperatura y humedad relativa, la humedad del suelo, la iluminación y la calidad del aire mediante la implementación de un conjunto de sensores electrónicos capaces de leerlas, como el DHT11, HL-69, LDR y MQ-135, mientras que la información recolectada es procesada por el microcontrolador ESP32 y presentada al usuario mediante una pantalla LCD.

Además del desarrollo del sistema de monitoreo, el proyecto aplica los principios de la Programación Orientada a Objetos (POO), implementando conceptos como abstracción, encapsulamiento, herencia y polimorfismo para lograr una estructura de software modular, organizada y mantenible en el tiempo. De esta manera se realiza una integración de conocimientos en electrónica, programación y sistemas embebidos para el desarrollo de una solución funcional orientada al monitoreo de variables ambientales.

VI. Marco Teórico

A. Estación Meteorológica

Una estación meteorológica se define formalmente como un emplazamiento técnico dotado de una variedad de instrumentos de medición, con el fin de observar, cuantificar, registrar y transmitir de manera sistemática los diferentes parámetros de las variables termodinámicas y cinéticas que configuran el estado físico del sistema atmosférico local [1].

En la ingeniería de sistemas embebidos se implementa la variante denominada Estación Meteorológica Automatizada. A diferencia de las estaciones analógicas convencionales que dependen enteramente de la lectura visual humana en intervalos temporales fijos, estas estaciones ejecutan de manera continua y autónoma los ciclos de muestreo, acondicionamiento de señal, almacenamiento digital y procesamiento analítico de datos climatológicos sin requerir intervención constante de operarios de campo [1].

La estructura lógica y funcional de una estación automatizada digital se compone de tres bloques arquitectónicos interconectados que abarcan un subsistema de transducción y adquisición, un subsistema central de procesamiento y un subsistema de interfaz y salida. El primero, constituido por una red de sensores electrónicos, transforman fenómenos físicos meteorológicos en señales eléctricas variables; el segundo, integrado por una unidad de microcontrolador que gestiona los tiempos de muestreo a través de temporizadores internos, ejecuta los algoritmos de filtrado digital y computa los promedios matemáticos requeridos para uniformizar los datos; y el tercero, canaliza la información procesada hacia módulos de almacenamiento masivo o visualizadores locales en tiempo real [2][3].

B. Programación Orientada a Objetos (POO)

POO es un paradigma de programación que utiliza representaciones de elementos de la vida real, llamados “objetos” para resolver problemas y desarrollar aplicaciones informáticas. Dichos objetos poseen atributos (características) y métodos (acciones o comportamientos) propios que los definen. [4]

La POO se diferencia de la programación estructurada cotidiana porque se fundamenta en 4 pilares:

- **Herencia:** Permite que una clase (hija) herede características de otra clase (padre), extendiendo la funcionalidad de una clase sin tener que reescribir el código principal, favoreciendo a su reutilización.
- **Encapsulamiento:** Protege los datos que conforman una clase y controla su acceso a través de métodos específicos (Setters y Getters) que se definen como públicos, protegidos o privados en función de su alcance.
- **Polimorfismo:** Permite que un mismo método tenga comportamientos diferentes dependiendo de la clase que lo utilice y los atributos que le sean entregados.
- **Abstracción:** Modela solo los comportamientos esenciales para el funcionamiento general de un objeto, omitiendo u ofuscando los detalles internos o complejos de funcionamiento.

En este paradigma, los programas se construyen mediante la interacción de objetos, los cuales son instancias de clases; mejorando la modularidad, la gestión



de memoria disponible, la organización y la reutilización del código a largo plazo. [4][5]

Una clase describe la esencia de un objeto, ya que es la abstracción que abarca los atributos, operaciones, relaciones y semántica comunes en un conjunto de objetos.

C. Programación Modular

Es un paradigma de desarrollo de software que consiste en dividir un programa o proyecto grande en componentes o módulos independientes, cada uno encargado de realizar una función específica dentro del sistema, que se interrelacionan y permite organizar el código de manera más estructurada, facilitando su comprensión, mantenimiento y reutilización en el futuro. Cada módulo puede desarrollarse, probarse y modificarse de forma independiente, lo que contribuye a mejorar la calidad y eficiencia del software, así como a evitar errores de lógica globales. [5][6]

En el lenguaje C++, su implementa comúnmente se realiza mediante la separación del código en dos tipos de archivos, los archivos de cabecera o librerías (".h") y los archivos fuente o de implementación (".cpp"). Los archivos de cabecera contienen las declaraciones de clases, funciones y métodos, mientras que los archivos de implementación almacenan la lógica y el comportamiento de dichos elementos. Esta estructura favorece la organización del proyecto y permite que distintos módulos interactúen entre sí de manera ordenada. [6][7]

D. Microcontrolador

Una MCU (Unidad de Microcontrolador) es un dispositivo constituido por un único circuito integrado capaz de ejecutar múltiples tareas siguiendo una secuencia lógica de instrucciones programadas y de interactuar con su entorno mediante pines destinados a dispositivos periféricos de entrada y salida. [8]

En comparación con los ordenadores, PLC u otros controladores programables, los microcontroladores destacan por su reducido tamaño, bajo consumo energético y bajo costo comercial. Adicionalmente su arquitectura flexible les permite ser utilizados para aplicaciones en áreas como la robótica, la electrónica, la industria automotriz, el IoT, la biomédica y los sistemas de navegación espacial, entre otros. [3]

Debido a su limitada capacidad de procesamiento, una de las principales restricciones de estos dispositivos

es el uso de la memoria disponible, la cual se encarga tanto del almacenamiento de datos como de su gestión mediante procesos de lectura y escritura.

E. Arduino

Una de las plataformas de desarrollo electrónico y cadena que abarca los microcontroladores más ampliamente conocidos y con la mayor cantidad de documentación disponible es el Arduino, un software y hardware Open Source (de código abierto) que ha democratizado el acceso al desarrollo de proyectos electrónicos y sistemas embebidos, permitiendo a usuarios analizar, modificar y distribuir recursos libremente. [9]

Creada en 2005 por un equipo del Instituto de Diseño Interactivo de Ivrea, Arduino está conformado por una placa electrónica con periféricos de entrada y salida, un microcontrolador ATmega pertenecen a la familia AVR (generalmente ATmega328) y un entorno de desarrollo integrado (IDE) que permite escribir, compilar y cargar código al microcontrolador vía USB, facilitando considerablemente la programación de dispositivos electrónicos. [9][10]

Arduino cuenta con una amplia variedad de modelos adaptados a las diferentes necesidades de los usuarios y sus aplicaciones, siendo los más utilizados el Arduino UNO, ampliamente empleado en proyectos educativos; el Arduino Mega, diseñado para proyectos más complejos que requieren una mayor cantidad de pines y memoria; y el Arduino Nano, utilizado en proyectos compactos y aplicaciones de IoT, entre muchas otros modelos.

Un complemento importante para el desarrollo de proyectos con esta MCU es Tinkercad. Desarrollada por Autodesk, esta plataforma en línea permite diseñar y simular circuitos electrónicos utilizando microcontroladores como Arduino y Microbit. Su interfaz gráfica intuitiva y amplia base de componentes preconfigurados la convierten en un referente para aficionados y principiantes. [11]

F. ESP32

Otro microcontrolador ampliamente conocido y versátil es el ESP32, desarrollado por Espressif Systems para aplicaciones de IoT, sistemas embebidos y dispositivos electrónicos de bajo consumo energético. Este microcontrolador incorpora conectividad WiFi y Bluetooth, lo que le permite comunicarse con una gran



variedad de dispositivos y módulos mediante interfaces como SPI e I2C, ya sea funcionando de manera independiente o como dispositivo esclavo de otra MCU principal. [12][13]

De acuerdo con su documentación oficial, el nivel de integración de los componentes (interruptores de antena, amplificadores y módulos) les permite a sus desarrolladores su implementación en sistemas precisos y compactos, lo que facilita significativamente su incorporación en circuitos impresos (PCB). [12]

Además, la arquitectura del ESP32 le permite ejecutar programas desarrollados en diferentes entornos de programación, tanto de código abierto como Arduino IDE, ampliamente empleado por su facilidad de uso y gran disponibilidad de librerías, como el framework oficial ESP-IDF proporcionado por Espressif Systems, que ofrece un mayor control sobre los recursos internos y funcionalidades avanzadas.

G. Sensores

Son dispositivos electrónicos diseñados para detectar cambios y estímulos en el entorno y transformarlos en señales digitales, las cuales son convertidas en impulsos eléctricos para que puedan ser leídas y procesadas por sistemas electrónicos y microcontroladores automatizando sistemas complejos [14]. Su implementación resulta fundamental en el desarrollo del presente proyecto debido a la necesidad de monitorear variables ambientales en tiempo real.

H. Sensor de Humedad del Suelo (HL-69)

El sensor capacitivo de humedad es un dispositivo utilizado para medir el nivel de humedad presente en un terreno mediante la variación de capacitancia producida por los cambios en el contenido de agua del suelo [15]. A diferencia de los sensores resistivos convencionales, los sensores capacitivos presentan una mayor resistencia a la corrosión y una mejor estabilidad durante su funcionamiento, siendo empleado para monitorear condiciones en el suelo y optimizar controles de riego.

I. Sensor de Temperatura y Humedad Relativa (DHT-11)

Es un sensor digital de bajo costo diseñado para medir temperatura y humedad relativa del ambiente, integrando tanto un sensor capacitivo de humedad como un termistor, permitiendo entregar datos mediante una

señal digital [16]. Su rango de operación es oportuno ya que lo hace compatible con microcontroladores como Arduino y ESP32, así como su facilidad de implementación y capacidad para medir más de una variable ambiental.

J. Sensor LDR

Está compuesto por una resistencia dependiente de luz (LDR) cuyo valor resistivo varía en función de la intensidad lumínica que perciba. A mayor cantidad de luz incidente en el ambiente, menor será el valor de su resistencia eléctrica [17]. Sus características lo hacen ampliamente utilizados en monitoreo ambiental y dispositivos de automatización de entornos.

K. Sensor de Calidad del Aire (MQ-135)

Es un sensor semiconductor utilizado para detectar diferentes gases peligrosos presentes en el aire, entre los cuales está el dióxido de carbono (CO_2), amoníaco (NH_3), benceno, humo y compuestos orgánicos volátiles (VOCs) [18]. Su aplicación principal está estrechamente relacionada con sistemas de monitoreo para garantizar la calidad e idoneidad del aire y la detección de posibles contaminantes ambientales, complementando significativamente el desarrollo de proyectos complejos.

L. Pantalla de Cristal Líquido (LCD)

Es un dispositivo de visualización optoelectrónico plano fundamentado en la manipulación controlada de las propiedades ópticas de los cristales líquidos en presencia de campos eléctricos. Estos compuestos químicos moleculares poseen un estado físico intermedio entre el líquido convencional y el sólido cristalino.

En una pantalla LCD nemática de torsión, el material se encapsula entre dos filtros polarizadores cuyos ejes de transmisión de luz están desfasados a 90 grados entre sí. Al no aplicar voltaje, la geometría interna de las placas obliga a las moléculas a alinearse en una trayectoria helicoidal, la cual rota la polarización de la luz incidente permitiendo que atraviese el segundo polarizador y el píxel se observe iluminado. Cuando se aplica un voltaje a través de los electrodos transparentes de óxido de indio y estaño, se genera un campo eléctrico que induce el alineamiento de los dipolos moleculares de forma paralela al campo, destruyendo la estructura helicoidal. Como consecuencia, la luz ya no rota su eje de



polarización, es absorbida por el segundo filtro óptico y el píxel toma una apariencia oscura [20].

En sistemas embebidos de medición, la visualización de variables se realiza de forma habitual mediante módulos LCD alfanuméricos monocromáticos, siendo el formato de 16 columnas por 2 filas uno de los estándares más comunes de la industria. Cada bloque de carácter está físicamente constituido por una matriz interna de píxeles, normalmente de 5 por 8 puntos. Para evitar que el microcontrolador deba gestionar individualmente la conmutación eléctrica, la pantalla integra un controlador específico (generalmente el HD44780).

M. Módulo de Comunicación I2C

El bus I2C es un protocolo de comunicación serie, síncrono, de tipo semidúplex y orientado a bytes, desarrollado por la división de semiconductores de Philips, NXP Semiconductors [20]. Su concepción responde a la necesidad de intercomunicar múltiples circuitos integrados de procesamiento y periféricos dentro de una misma tarjeta de circuito impreso, optimizando el espacio físico y reduciendo la complejidad de este.

El bus se compone de dos líneas físicas denominadas SDA (para la transmisión secuencial de bits de datos) y SCL (para transportar los pulsos de reloj generados por el dispositivo maestro), además de su conexión VCC y GND. A nivel de hardware, ambas líneas están configuradas con etapas de salida de colector abierto, lo que impide que ocurran cortocircuitos por contención de bus cuando se intentan sobrescribir estados lógicos opuestos simultáneamente. Para garantizar el correcto funcionamiento del bus y establecer los niveles lógicos por defecto, se requiere conectar obligatoriamente dos resistencias de elevación vinculadas a la línea de alimentación eléctrica [20].

La arquitectura operativa de I2C se basa en un esquema jerárquico de maestro y esclavo, donde el dispositivo maestro posee el control absoluto del bus, siendo el único facultado para iniciar las transferencias de datos, fijar la velocidad de conmutación y generar los estados específicos de inicio y parada [16]; y los dispositivos esclavos permanecen en un estado pasivo de escucha, respondiendo exclusivamente cuando su dirección asignada de 7 bits es transmitida por el bus [17].

N. Repositorio en Línea GitHub

GitHub es una plataforma de desarrollo colaborativo basada en la nube creada por Microsoft, la cual permite a los usuarios almacenar, gestionar y compartir proyectos de software y complementar su desarrollo mediante el uso del sistema de control de versiones Git. Esta herramienta facilita el seguimiento de cambios realizados en los archivos de un proyecto, permitiendo que varios desarrolladores trabajen simultáneamente sobre el mismo código de manera organizada y eficiente.

Además de servir como repositorio remoto, GitHub incorpora herramientas orientadas al trabajo colaborativo, tales como gestión de ramas, revisión de código, seguimiento de errores (*issues*) y documentación de proyectos. Estas funcionalidades permiten mejorar la organización y el mantenimiento de proyectos grandes o complejos, facilitando la integración continua y el desarrollo simultáneo de diferentes módulos del sistema.

O. Sistema de Control de Versiones (Git)

Git es un sistema de control de versiones de código abierto diseñado para registrar de manera cronológica los cambios realizados en el código fuente de un proyecto de software [21]. A diferencia de otras herramientas, su arquitectura permite a cada desarrollador poseer una copia local, exacta y completa de todo el historial de cambios en un repositorio, proporcionando registros de “back up” frente a pérdidas de información y agilizando operaciones monótonas de desarrollo.

Su flujo de trabajo se organiza en tres secciones lógicas para los archivos según su estado: El directorio de trabajo, el área de preparación o staging area y el repositorio local [21]. Cualquier modificación realizada en el código se encuentra inicialmente en el directorio de trabajo; posteriormente, el programador selecciona qué cambios incluir y los traslada al área de preparación; finalmente, tras una confirmación (*commit*), las modificaciones se guardan de forma permanente y segura en la base de datos del repositorio local “.git”.

El uso de Git resulta fundamental para gestionar de manera segura y organizada el desarrollo y evolución de software de sistemas embebidos y proyectos de gran escala, así como la integración de distintos módulos al sistema [22], permitiendo crear ramas que funcionan como líneas de desarrollo independientes donde es posible implementar, probar y depurar funcionalidades



específicas sin comprometer la estabilidad del código principal.

VII. Metodología

El desarrollo de la Miniestación Meteorológica Automatizada se llevó a cabo mediante fases de diseño e implementación por etapas, fundamentada en los principios de POO y la integración coherente entre el hardware y el software del sistema.

Como punto de partida, se realizó un análisis exhaustivo de los requerimientos del sistema, en el cual se identificaron 5 variables ambientales objeto de monitoreo: temperatura y humedad ambiental, humedad del suelo, intensidad lumínica y calidad del aire, así como del microcontrolador encargado de la lectura de las mediciones tomadas por los dispositivos de monitoreo.

A partir de esta necesidad, se seleccionó un microcontrolador ESP32 DevKit-V1, así como los sensores necesarios para las mediciones de dichas variables ambientales, entre los que se tomaron en cuenta los sensores DHT11, HL-69, LDR y MQ-135 por su asequibilidad y sencilla programación, y una pantalla LCD junto a un módulo I2C para la visualización local de los datos recolectados en tiempo real.

Una vez definidos los requerimientos, se realizó el diseño estructural del sistema mediante la elaboración de diagramas UML que evidenciara la incorporación de POO en el proyecto. Para ello, se estableció una clase abstracta denominada *Sensor* con el propósito de definir una interfaz común para todos los dispositivos de medición, una clase particular para la pantalla LCD y una clase maestra *EstacionMetereologica* encargada de la gestión conjunta de todos los sensores, lecturas, toma de datos e impresiones.

A partir de la clase *Sensor* se implementaron las clases derivadas *SensorAmbiental*, *SensorSuelo*, *SensorLuz* y *SensorAire* con sus respectivos métodos y constructores sobre escritos haciendo uso de los mecanismos de herencia y polimorfismo propios de POO. A su vez, la clase *EstacionMetereologica*, se encargaría de la administración general del sistema y del manejo dinámico de memoria para optimizar el control y mantenimiento del sistema.

Concluido el diseño, la fase de implementación requirió la incorporación de un software capaz de comunicarse con el microcontrolador, para lo que se

utilizó el editor de código gratuito de Microsoft, Visual Studio Code, incorporando la extensión PlatformIO de comunicación con sistemas embebidos, y el framework de Arduino IDE para facilitar el flujo de trabajo durante la programación del proyecto.

Con el fin de favorecer el mantenimiento del código, el proyecto se organizó en archivos de encabezado (.h) y archivos de implementación (.cpp) y se vinculó a un repositorio web en GitHub con un control de versiones a través de Git, de manera que se pudieran contrastar los cambios en el tiempo y crear nuevas ramas (*branches*) en caso de cambios experimentales o de estructuración.

Posteriormente, se realizó la integración física del hardware y sus conexiones a través de una placa de circuito integrado (PCB) conforme a los requerimientos electrónicos de cada dispositivo vinculado. Tras la integración de cada sensor y la pantalla al circuito de la ESP32, se adicionaron 2 pulsadores normalmente abiertos que permitirían el desplazamiento de la información mostrada por la LCD a través de un array.

Para complementar la presentación del proyecto en una escenografía agradable, se preparó una maqueta física compuesta en su mayoría por material reciclable o elementos sobrantes de proyectos anteriores como lo fue el caso de la base en impresión 3D. Se utilizó una tabla para el techo y soportes hechos de escoba, de manera que dentro de este recinto pudiera ubicarse una planta pequeña y los sensores sobresalieran en la parte superior.

Finalmente, se llevó a cabo una etapa de pruebas y validación la cual comprendió, en primer lugar, pruebas individuales de cada dispositivo para verificar su funcionamiento autónomo, y posteriormente, pruebas de integración orientadas a comprobar la correcta interacción entre los distintos componentes del sistema, evaluando la lectura de datos, las actualizaciones en la pantalla LCD y el comportamiento del programa bajo diversas condiciones ambientales, con el fin de poder garantizar la confiabilidad y estabilidad del sistema.

Cabe destacar que se realizaron cambios después de cada fase para solucionar imprevistos o elementos que quedaron por profundizar a posteriori, como lo fue las modificaciones en los diagramas UML, implementación de una documentación Doxygen de los métodos y clases del sistema, la incorporación de comentarios para aclarar el funcionamiento dentro del código fuente y la redacción y traducción del informe final a LaTeX.

VIII. Análisis de Resultados

Durante la fase de testeo, las pruebas realizadas indicaron un correcto funcionamiento de los sensores implementados y comunicación entre el microcontrolador y la pantalla de visualización en la miniestación meteorológica.

El sensor DHT11 permitió obtener lecturas de temperatura y humedad relativas cercanas a las condiciones del entorno, mostrando una respuesta estable y consistente durante las pruebas teniendo en cuenta su condición como un sensor exclusivamente digital poco preciso.

Análogamente, el sensor HL-69 detectó variaciones en la humedad del suelo en tiempo real al entrar en contacto con los diferentes niveles de humedad a los que fue expuesto, permitiendo identificar valores intermedios entre el suelo seco y húmedo y picos de humedad.

Al contrario, respecto al sensor de luz (LDR), se observó que las lecturas presentaban únicamente dos estados posibles: Ausencia de luz o iluminación máxima. Este comportamiento se debe principalmente a que la referencia adquirida solo cuenta con una salida digital (D0) en vez de una salida analógica. Aunque el comparador interno del sensor solo entrega valores lógicos *High* o *Low*, siendo imposible medir la intensidad luminosa de forma gradual, sigue siendo útil para detectar cambios drásticos en la iluminación del ambiente.

Por otra parte, el sensor MQ-135 respondió satisfactoriamente a las variaciones de partes por millón (PPM) producto de la acción agentes contaminantes como el humo, mostrando lecturas analógicas coherentes y comparando la calidad del aire expuesto tras la prueba. Si bien los valores obtenidos no representan directamente una concentración específica de gases, permitieron evidenciar la capacidad del sistema para monitorear condiciones ambientales relacionadas con la calidad del aire.

Asimismo, las pruebas individuales realizadas para la pantalla LCD mostraron la información obtenida por los sensores a tiempo real, así como durante las pruebas de integración donde se realizaron impresiones a intervalos de tiempo regulares y la modificación de la información al interactuar con pulsadores, facilitando la visualización de las variables monitoreadas de manera local.

Por otra parte, la implementación de POO mediante clases, herencia, encapsulamiento y polimorfismo en el

proyecto permitió desarrollar un código modular, organizado y fácil de editar y mantener incluso ante la incorporación de nuevos sensores o dispositivos al sistema para su enriquecimiento.

En general, los resultados obtenidos demostraron que la miniestación meteorológica fue capaz de adquirir, procesar y visualizar la mayor parte de la información ambiental de manera satisfactoria, sin embargo, se identificaron el uso de un sensor LDR con salida analógica (A0), una calibración más precisa del sensor MQ-135 y el cambio del sensor DHT11 por el DHT22 como puntos de mejora para obtener mediciones más exactas y coherentes con las variaciones ambientales.

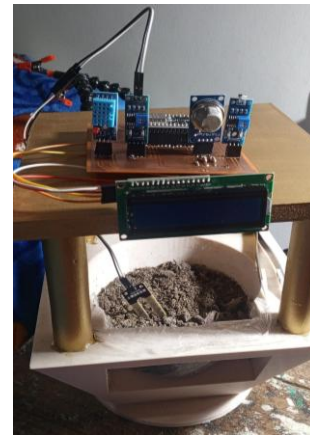


Fig. 1 Prototipo de la Miniestación Meteorológica funcional en la maqueta física mostrando lecturas del *SensorAmbiental* (temperatura y humedad relativa)

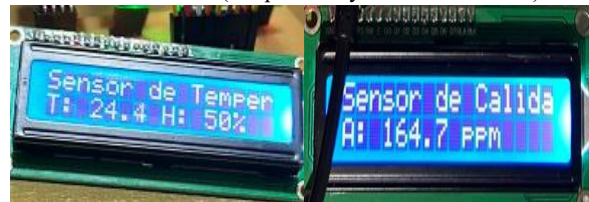


Fig. 2 Mediciones de *SensorAmbiental* y *SensorAire* de la Miniestación Meteorológica durante una prueba de integración

IX. Conclusiones

El desarrollo de la miniestación meteorológica automatizada permitió monitorear variables ambientales y sus fluctuaciones en el tiempo debido a cambios en las condiciones del entorno satisfactoriamente, a través de la integración de sensores de temperatura y humedad relativa, humedad del suelo, iluminación y calidad del aire en un programa o plataforma de monitoreo soportado por el microcontrolador ESP32, así como mostrar dichas



lecturas de datos visualmente en tiempo real a través de una pantalla LCD, facilitando la supervisión localmente.

Asimismo, la implementación de la Programación Orientada a Objetos contribuyó significativamente a la organización y estructuración del software, permitiendo aplicar conceptos como abstracción, encapsulamiento, herencia y polimorfismo en la construcción de este, lo cual favoreció el modularidad del código, su mantenimiento, comprensión y posible ampliación para futuras versiones del proyecto.

Por su parte, la combinación del microcontrolador ESP32 con el uso de materiales reciclables para la construcción de la maqueta física, dio como resultado un prototipo que combina eficiencia, bajo costo y sostenibilidad. Este diseño demuestra que la optimización de los recursos materiales mediante la reutilización de componentes y estructuras es clave para la elaboración de soluciones tecnológicas funcionales que minimizan el gasto presupuestario y su huella ambiental.

No obstante, las pruebas realizadas permitieron identificar las capacidades y limitaciones de los sensores utilizados. En particular, se observó que el sensor LDR únicamente permitió detectar estados de presencia o ausencia de luz, mientras que el sensor DHT11 proporcionó una referencia general de los valores de temperatura y humedad relativa, más carecieron de precisión en el entorno. Esto evidencia la importancia de la correcta selección y configuración de los dispositivos de medición para obtener resultados más precisos.

Finalmente, el proyecto permitió fortalecer los conocimientos adquiridos en electrónica analógica, programación de microcontroladores y diseño de proyectos orientado a objetos, integrando componentes de hardware y software en una solución tecnológica funcional y asequible que puede mejorarse a través de la implementación de procesos de calibración más avanzados, el uso de sensores con mayor precisión y la incorporación de sistemas de almacenamiento o transmisión de datos de manera inalámbrica para ampliar las funcionalidades de la estación meteorológica.

X. Referencias

- [1] Organización Meteorológica Mundial, *Guía de instrumentos y métodos de observación meteorológicos*, OMM-N.º 8, 7.ª ed. Ginebra, Suiza: Secretaría de la OMM, 2008.
- [2] T. L. Floyd, *Dispositivos electrónicos*, 8.ª ed. México D.F., Pearson Educación, 2008, pp. 110-115.
- [3] T. L. Floyd, *Fundamentos de sistemas digitales*, [En línea]. Disponible en: https://gc.scalahed.com/recursos/files/r161r/w25733w/fundamentos-de-sistemas_compressed.pdf. [Accedido: 31-may-2026].
- [4] Juan Carlos Moreno, "Programación Orientada a Objetos (MF0227_3)", Spain: Rama Editorial, 2016. [En línea]. Disponible: <https://www-digitaliaphublishing-com.unipamplona.basesdedatosezproxy.com/a/110026>. [Accedido: 31-may-2026].
- [5] P. J. Ponce de León Amador, y E. Saquete Boro, "Introducción a la programación orientada a objetos", Spain: Publicaciones de la Universidad de Alicante, 2006. [En línea]. Disponible: <https://www-digitaliaphublishing-com.unipamplona.basesdedatosezproxy.com/a/701>. [Accedido: 31-may-2026].
- [6] P. J. Deitel y H. M. Deitel, C++: *Cómo programar*, 10.ª ed. Ciudad de México, México: Pearson Educación.
- [7] B. Stroustrup, *The C++ Programming Language*, 4th ed. Boston, MA, EE. UU.: Addison-Wesley Professional.
- [8] Néstor Fernando Penagos Quintero, "Microcontroladores: Microchip, Atmel, NXP Freescale y Texas Instruments. Pasos para una programación con éxito", Universidad Piloto de Colombia, 2022. [En línea]. Disponible: <https://www-digitaliaphublishing-com.unipamplona.basesdedatosezproxy.com/a/119483>. [Accedido: 31-may-2026].
- [9] Maxwell Arbey Salazar Guilcamagua, y Paola Alexandra Portero Donoso, "Introducción a la programación con Arduino: Guía básica con ejemplos prácticos", Editorial Exced, 2025. [En línea]. Disponible: <https://www-digitaliaphublishing-com.unipamplona.basesdedatosezproxy.com/a/180237>. [Accedido: 31-may-2026].
- [10] Eugenio López, "Arduino. Guía práctica de fundamentos y simulación: Guía práctica de fundamentos y simulación", Rama Editorial, 2016. [En línea]. Disponible: <https://www-digitaliaphublishing-com.unipamplona.basesdedatosezproxy.com/a/110057>. [Accedido: 31-may-2026].
- [11] Autodesk, "Basics of Arduino – Tinkercad", Tinkercad, [En línea]. Disponible en: <https://www.tinkercad.com/projects/Basics-of-Arduino-TINKERCAD>. [Accedido: 31-may-2026].
- [12] Espressif Systems, "ESP32 Series", Espressif Systems, [En línea]. Disponible en: <https://www.espressif.com/en/products/socs/esp32>. [Accedido: 31-may-2026].
- [13] Espressif Systems, "ESP32 Wi-Fi & Bluetooth Modules", Espressif Systems, [En línea]. Disponible en:



<https://www.espressif.com/en/products/modules/esp32>.

[Accedido: 31-may-2026].

- [14] SDI Industrial, “Sensores: definición y funcionamiento”, *SDI Industrial*, [En línea]. Disponible en: <https://sdindustrial.com.mx/blog/sensores/>. [Accedido: 31-may-2026].
- [15] EOS Data Analytics, “Sensores de humedad del suelo”, *EOSDA*, [En línea]. Disponible en: <https://eos.com/es/blog/sensores-de-humedad-del-suelo/>. [Accedido: 31-may-2026].
- [16] Naylamp Mechatronics, “Sensor de temperatura y humedad relativa DHT11”, *Naylamp Mechatronics*, [En línea]. Disponible en: <https://naylampmechatronics.com/sensores-temperatura-y-humedad/57-sensor-de-temperatura-y-humedad-relativa-dht11.html>. [Accedido: 31-may-2026].
- [17] ALLELCO, “¿Qué es un LDR y cómo funciona en electrónica?”, *ALLELCO Electronics*, [En línea]. Disponible en: <https://www.allelcoelec.es/blog/what-is-an-ldr-and-how-it-works-in-electronics.html>. [Accedido: 31-may-2026].
- [18] Instructables, “Calidad de aire – Sensor MQ-135”, *Instructables*, [En línea]. Disponible en: <https://www.instructables.com/Calidad-De-Air-Sensor-Mq-135/>. [Accedido: 31-may-2026].
- [19] NXP Semiconductors, “I2C-bus specification and user manual”, Rev. 6, Doc. UM10204, abr. 2014. [En línea]. Disponible en: <https://www.nxp.com/docs/en/user-manual/UM10204.pdf>. [Accedido: 31-may-2026].
- [20] Texas Instruments, “PCF8574 Remote 8-Bit I/O Expander for I2C Bus”, Hoja de datos SCPS068J, Dallas, TX, EE. UU.
- [21] S. Chacon y B. Straub, “Pro Git”, 2.^a ed. Nueva York, EE. UU. *Apress*. [En línea]. Disponible en: <https://git-scm.com/book/es/v2>. [Accedido: 31-may-2026].
- [22] E. Ribeiro, *Git for Embedded System Developers: Version Control for Hardware and Firmware*. Londres, Reino Unido. *Packt Publishing*, 2019, pp. 88-102.



SC-CER96940



“Formando nuevas generaciones con sello de excelencia comprometidos con la transformación social de las regiones y un país en paz”

Universidad de Pamplona
Pamplona - Norte de Santander - Colombia
Tels: (+57) 3153429495 - 3160244475
www.unipamplona.edu.co



XI. Anexos

TABLA 1
FICHA TÉCNICA DEL SENSOR DHT-11

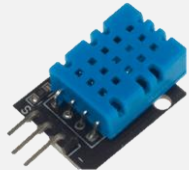
Característica	Valor
Ilustración	
Tipo de sensor	Temperatura y humedad relativa
Voltaje de operación	3.3 V – 5 V DC
Corriente de operación	0.3 mA (medición)
Corriente en espera	60 μ A
Potencia aproximada	1.5 mW
Rango de temperatura	0 °C a 50 °C
Precisión temperatura	± 2 °C
Rango de humedad	20 % a 90 % HR
Precisión humedad	± 5 % HR
Tiempo de respuesta	Menor a 6 segundos
Tipo de salida	Digital

TABLA 2
FICHA TÉCNICA DEL SENSOR LDR

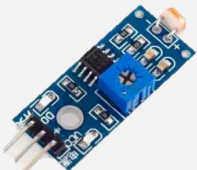
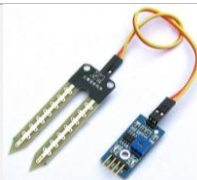
Característica	Valor
Ilustración	
Tipo de sensor	Resistencia dependiente de la luz (LDR)
Variable medida	Intensidad lumínica
Voltaje de operación	Depende del circuito
Corriente de operación	Muy baja (μ A a mA)
Potencia aproximada	Menor a 100 mW
Tipo de salida	Analógica
Resistencia en oscuridad	Hasta varios M Ω
Resistencia con luz intensa	Cientos de Ω a pocos k Ω
Tiempo de respuesta	20 ms – 100 ms
Ángulo de detección	120° – 180°

TABLA 3
FICHA TÉCNICA DEL SENSOR HC-69

Característica	Descripción
Ilustración	
Tipo de sensor	Sensor capacitivo de humedad del suelo
Variable medida	Humedad del suelo
Voltaje de operación	3.3 V a 5.5 V DC
Corriente de operación	Menor a 5 mA
Potencia aproximada	Menor a 25 mW
Tipo de salida	Analógica
Principio de funcionamiento	Variación de la capacitancia según el contenido de agua del suelo
Rango de detección	Desde suelo seco hasta suelo completamente húmedo
Precisión	Depende de la calibración y del tipo de suelo utilizado
Material sensible	Placa capacitiva anticorrosión
Resistencia a la corrosión	Alta
Tiempo de respuesta	Rápido, en tiempo real
Ventajas principales	Mayor durabilidad, estabilidad de medición y resistencia a la corrosión en comparación con sensores resistivos

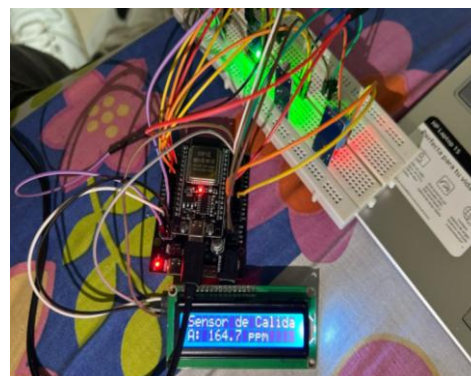


Fig. 3 Desarrollo de las conexiones en protoboard para el primer prototipo de la Miniestación Meteorológica

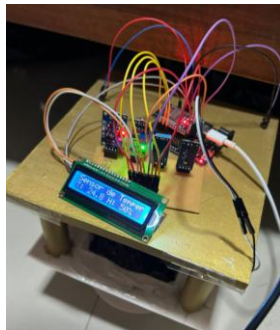
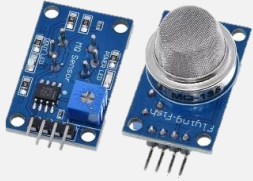


Fig. 4 Prototipo funcional de la Miniestación Meteorológica en la maqueta física mostrando lecturas del *SensorAmbiental* (temperatura y humedad relativa)



Fig. 5 Código QR de acceso al repositorio de GitHub del proyecto

TABLA 4
FICHA TÉCNICA DEL SENSOR MQ-135

Característica	Valor
Ilustración	
Tipo de sensor	Calidad del aire
Voltaje de operación	5 V DC
Corriente de operación	≈ 150 mA
Potencia del calentador	≈ 800 mW
Tipo de salida	Analógica y digital
Gases detectados	CO ₂ , NH ₃ , H ₂ S, benceno, alcohol, humo y VOCs
Rango de detección	10 ppm – 1000 ppm
Temperatura de operación	-10 °C a 50 °C
Humedad de operación	Hasta 95 % HR
Tiempo de precalentamiento	24 – 48 h
Sensibilidad	Ajustable mediante calibración